

Задача 1 (2 балла)

Рассмотрим следующую программу:

```
#include <iostream>

int main() {
    for (int i = 0; i < 300; ++i) {
        std::cout << i << ' ' << i*12345678 << std::endl;
    }
}
```

После компиляции на g++ с параметром -O2 и запуска выяснилось, что цикл в этой программе работает бесконечно. Объясните подробно: что произошло и почему?

Задача 2 (2 балла)

Рассмотрим код:

```
int& f() {
    int* p = new int;
    return *p;
}

int main() {
    int& x = f();
    delete &x;
}
```

- 1) Вызовет ли этот код SE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран.
- 2) Ответьте на те же вопросы, если в обоих случаях вместо int& написать просто int.

Задача 3 (2 балла)

Рассмотрим код:

```
#include <iostream>

void f(int**) {
    std::cout << 1;
}

void f(int(*)[5]) {
    std::cout << 2;
}

void f(int*[5]) {
    std::cout << 3;
}

int main() {
    int a[5];
    f(&a);
    int* b = a;
    f(&b);
}
```

- 1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран.
- 2) Ответьте на те же вопросы, если удалить третье объявление f вместе с определением (оставить только первое и второе).

Задача 4 (3 балла)

Рассмотрим код:

```
#include <iostream>

int main() {
    int x = 0;
    const int& y = x;
    ++x;
    int z = y;
    ++z;
    int t = const_cast<int&>(y);
    --t;
    std::cout << x << y << z << t;
}
```

- 1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран.
- 2) Ответьте на те же вопросы, если переменную z объявить как int& вместо int.
- 3) Ответьте на те же вопросы, если переменную z оставить как int, но переменную t объявить как int& вместо int.

Задача 5 (2 балла)

Рассмотрим программу:

```
#include <iostream>

struct A {
    int x = 0;
    int& r = x;
    x = 1;

    A(const int y) {
        r = y;
    }
    ~A() {
        r = x = 0;
    }
};

int main() {
    A a(5);
    A& aa = a;
    a = aa;
}
```

Какие ошибки компиляции возникнут в этой программе? Про каждую объясните, из-за чего она возникнет.

Задача 6 (2 балла)

Рассмотрим код:

```
#include <iostream>

struct A {
    void f() {}
};

struct B {
    A a;
    int i; // 1
    char c; // 2
};

struct C: public A {
    char c;
    int i;
    void g() {}
};

int main() {
    std::cout << sizeof(A) << ' ' << sizeof(B) << ' ' << sizeof(C);
}
```

- 1) Будем считать, что `sizeof(char)` равно 1, `sizeof(int)` равно 4. Что выведется при запуске этого кода? Объясните каждое из чисел.
- 2) Изменится ли что-то, если поменять местами объявления `int i` и `char c` в структуре `B`? Объясните ответ.

Задача 7 (2 балла)

Рассмотрим код:

```
#include <iostream>
struct S {
    static int x;

    struct SS {
        float y = x;
    };

    SS f() /*const*/ {
        ++x;
        SS ss;
        std::cout << ss.y;
        return ss;
    }
};

int S::x = 1;

int main() {
    S s;
    S::SS ss;
    std::cout << ss.y << s.f().y;
}
```

- 1) Вызовет ли этот код `CE`, `RE`, `UB` или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет `UB`) и почему, во втором -- ответьте, что выведется на экран.
- 2) Ответьте на те же вопросы, если раскомментировать слово `const` в выделенном месте.

Задача 8 (2 балла)

Рассмотрим код:

```
#include <iostream>

struct A {
    A(int x) {
        std::cout << "A " << x;
    }
    ~A() {
        std::cout << "~A ";
    }
};

struct Base {
    A a{0};
    Base(int x): a(x) {
        std::cout << "Base";
    }
    ~Base() {
        std::cout << "~Base";
    }
};

struct Derived: Base {
    A b{0};
    Derived(int x, int y): /*Base(x),*/ b(y) {
        std::cout << "Derived";
    }
    ~Derived() {
        std::cout << "~Derived";
    }
};

int main() {
    Derived d(1, 2);
}
```

1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран.

2) Ответьте на те же вопросы, если раскомментировать выделенный фрагмент в списке инициализации.

Задача 9 (3 балла)

Рассмотрим следующий код:

```
#include <iostream>
class Granny {
protected:
    int g = 0;
    friend int main();
};

class Mom: private Granny {
protected:
    int m = 1;
    void f() {
        std::cout << g; // 1
    }
    friend class Son;
};

class Son: protected Mom {
public:
    int s = 2;
    void f() {
        std::cout << m << g; // 2
    }
};

int main() {
    Son s;
    std::cout << s.s << s.m << s.g; // 3
}
```

Про каждую из выделенных строк (1, 2 и 3) ответьте: возникнут ли в этой строке ошибки и что это будут за ошибки? Какие переменные будут доступны в каждой из этих строк, а какие недоступны, и почему?

Задача 10 (2 балла)

Рассмотрим код:

```
#include <iostream>
class Base {
private:
    virtual void do_run(double y) { std::cout << y; }
public:
    void run(double y) { do_run(y); }
};

class A : public Base {
public:
    void do_run(double y) override { std::cout << y+1; }
public:
    void run(int x) { Base::run(x); }
};

int main() {
    A a;
    a.run(0);
    Base& b = a;
    b.run(0);
}
```

- 1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран и почему.
- 2) Ответьте на те же вопросы, если в выделенной строке заменить слово public на private.

Задача 11 (2 балла)

Рассмотрим код:

```
#include <iostream>

template <typename T, typename U>
void f(T x, U y) {
    std::cout << 1 << '\n';
}
// template <>
void f(int x, int y) {
    std::cout << 2 << '\n';
}
template <typename T>
void f(T x, T y) {
    std::cout << 3 << '\n';
}
int main() {
    f(0, 0);
    f(0, 0.0);
    f(0.0, 0.0);
    f<int>(0, 0);
}
```

- 1) Что выведется на экран при его запуске? Объясните подробно, какая версия f выберется в каждом случае и почему. Если вы считаете, что возникнет ошибка компиляции, также поясните, почему она возникнет.
- 2) Что изменится, если раскомментировать строку `template <>?` Объясните подробно.

Задача 12 (3 балла)

Рассмотрим следующий код:

```
#include <iostream>
#include <type_traits>

template <typename T>
void f(std::conditional< std::is_class<T>, T&, T > value) {
    if (std::is_same<T, int>) {
        value *= 5;
    } else {
        value += "def";
    }
}

int main() {
    int x = 5;
    f<int>(x);
    std::string str = "abc";
    f<std::string>(str);

    std::cout << x << ' ' << str;
}
```

По задумке, функция f должна принимать объект по ссылке, если это string, и по значению, если это int. В результате должно было вывестись 5 abcdef. Но в реализации функции f допущены ошибки.

- 1) Объясните, какие ошибки компиляции вызовет этот код.
- 2) Про каждую из ошибок объясните, как стоило бы ее исправить, чтобы функция f компилировалась и работала как ожидалось.